

Методичка проєкту KompasMigracji / KompasCRM

Повний розгорнутий довідник з архітектури, модулів, ботів, платежів, бази даних і робочих процесів. Складено за результатами аудиту живого коду — не за застарілими README.

ПІДГОТОВЛЕНО ДЛЯ

Шеф Олександр

СТАН КОДОВОЇ БАЗИ

22 липня 2026

ПРОДАКШН

Vercel · push у main

Next.js 14

5 локалей

CRM · 140+ модулів

4 AI-боти

3 платіжні провайдери

Supabase / Postgres

Зміст

1. Що це за проєкт — за 60 секунд
2. Технологічний стек
3. Швидкий старт і команди
4. Карта репозиторію
5. Маршрутизація та middleware
6. Автентифікація, ролі та безпека
7. Публічний сайт (5 мов)
8. Адмінка / CRM — два паралельні світи
9. База даних — два клієнти, 31 міграція
10. AI-підсистеми — чотири боти та LifeOS
11. Платежі — три провайдери
12. Месенджери та вебхуки
13. Cron-завдання
14. Портали: клієнт, учасник, NPS
15. Тестування та CI
16. Змінні середовища (ENV)
17. Довідник бібліотек lib/
18. Рецепти типових задач
19. Граблі, на які вже наступали
20. Де шукати додаткову документацію

Посилання виду «§ 9» у тексті ведуть на відповідний розділ цього документа. Шляхи файлів подано відносно кореня репозиторію.

Що це за проєкт — за 60 секунд

KompasCRM (клієнтський бренд — **Kompas Migracji**) — моноліт на Next.js 14 для юридичної компанії DOMUS V, що обслуговує мігрантів у Польщі та ЄС. В одному застосунку живуть **три дуже різні поверхні**:

ПОВЕРХНЯ	ДЕ ЖИВЕ	ДЛЯ КОГО	ЩО РОБИТЬ
Публічний сайт	app/[locale]/*	Клієнти-мігранти	Маркетинг, прайси, AI-чатбот, збір лідів. 5 мов
CRM / Адмінка	app/admin/*	Співробітники	140+ модулів: ліди, угоди, справи, фінанси, HR, документи. Доступ за ролями (RBAC)
Спецсистеми	app/architect · portal · member · payment	Різні	LifeOS, портал справ клієнта, кабінет учасника профспілки, сторінки оплат

5

мов інтерфейсу (uk · pl · en · ru · rom)

4 + 1

AI-боти + система LifeOS

31

SQL-міграція бази даних

Під капотом: чотири незалежні AI-боти (чатбот сайту, Оракул для кандидатів, Оракул для роботодавців, Мілена-продажниця), оркестрація агентів God/Primus, три платіжні провайдери (Przelewy24, PayU, Stripe) і чотири месенджер-канали (WhatsApp, Telegram, Viber, Facebook).

КЛЮЧОВА ІДЕЯ

Це не «один сайт» — це три застосунки, склеєні одним middleware.ts . Майже кожна помилка новачка тут — редагування не того дерева маршрутів або використання не того клієнта БД.

Технологічний стек

ШАР	ТЕХНОЛОГІЯ	ПРИМІТКА
Фреймворк	Next.js 14 (App Router)	Мішаний TypeScript + JavaScript — навмисно, див. § 19
UI	React 18, Tailwind CSS 4 , framer-motion, lucide-react, recharts	Глас-морфізм адмінки — окремий <code>styles/glass.css</code>
Локалізація	next-intl	5 локалей: uk, pl, en, ru, rom; дефолт — uk
База даних	Supabase (Postgres)	Два клієнти доступу — raw <code>pg</code> і Supabase JS, НЕ взаємозамінні (§ 9)
Автентифікація	JWT у httpOnly-cookie (<code>jose</code>) + <code>bcryptjs</code> + TOTP 2FA	Cookie: <code>kompascrm_session</code>
AI	<code>ai</code> SDK + Gemini, Anthropic Claude, OpenAI	Різні боти — різні моделі (§ 10)
Платежі	Przelewy24 · PayU · Stripe	Пробуються по черзі (§ 11)
Пошта	SendGrid, nodemailer, imapflow	imapflow — вхідна IMAP-пошта
Пакети	pnpm	Тільки <code>pnpm</code> — не <code>npm</code> і не <code>yarn</code>
Хостинг	Vercel	Security-заголовки і крони — у <code>vercel.json</code>

Швидкий старт і команди

```
pnpm install      # встановити залежності
pnpm dev          # дев-сервер → http://localhost:3000
pnpm build        # продакшн-збірка
pnpm lint         # ESLint
pnpm typecheck    # tsc --noEmit – перевірка типів
pnpm test:unit    # Jest (юніт-тести)
pnpm test:e2e     # Playwright (сам підніме dev-сервер)
pnpm seed:agents  # засіяти таблиці агентів God/Primus
```

Запуск одного тесту

```
pnpm test:unit -- lib/__tests__/milena-bot.test.ts
pnpm test:unit -- -t "назва тесту"
pnpm test:e2e -- e2e/agents.spec.ts
```

НАЙВАЖЛИВІШЕ ПРАВИЛО ЗБІРКИ

`pnpm build` **не ловить помилки типів і літера** — у `next.config.mjs` навмисно ввімкнені `ignoreDuringBuilds` та `ignoreBuildErrors`. Зелена збірка ≠ робочий код. Перед комітом завжди окремо: `pnpm lint` і `pnpm typecheck`.

GitHub Actions CI — **заглушка** (`echo "Checks passed"`), що існує лише щоб розблокувати Vercel. Зелена галочка в CI нічого не доводить.

Карта репозиторію

```

KompasMigracji/
├─ app/
│   ├─ [locale]/      ← публічний сайт (uk/pl/en/ru/rom)
│   ├─ admin/         ← CRM: (panel) ~115 модулів + crm/ ~27 + login, setup, agents
│   ├─ api/           ← усі API-роути (admin, bot, chat, cron, god, orakul, payment...)
│   ├─ architect/     ← LifeOS-панель (тільки роль admin)
│   ├─ portal/        ← клієнтський портал справ (без логіна в CRM)
│   ├─ member/        ← кабінет учасника профспілки
│   ├─ payment/       ← сторінки оплат (без локального префікса!)
│   ├─ nps/           ← сторінка NPS-опитування
│   └─ architecture/ ← презентаційна сторінка архітектури
├─ components/        ← публічні компоненти + admin/ + lifeos/ + member/ + payment/
├─ lib/               ← вся бізнес-логіка (див. § 17)
├─ db/migrations/     ← SQL-міграції 001-031
├─ db/schema.sql      ← базова схема (старі kompas_* таблиці)
├─ supabase/*.sql     ← схеми, застосовані напряму в Supabase
├─ messages/         ← переклади {uk,pl,en,ru,rom}.json + admin/
├─ e2e/              ← Playwright-тести
├─ __tests__/ + lib/__tests__/ ← Jest-тести
├─ docs/             ← інструкції (ця методичка, crm_manual_uk, website_manual_uk)
├─ scripts/          ← утиліти; _archive/ – історичне сміття, НЕ зразок
├─ middleware.ts      ← маршрутизація + захист + rate limit (§ 5)
├─ i18n.ts           ← конфіг локалей
└─ vercel.json        ← security-заголовки + крони
  
```

Маршрутизація та middleware — серце системи

Файл `middleware.ts` — «диспетчер вокзалу»: кожен запит проходить крізь нього, і саме він вирішує, яке з трьох дерев відповідає.



Гілка JWT: невалідний токен → редірект `/admin/login` (для API — 401) · роль `member` → тільки `/admin/me` · `/architect` → лише роль `admin`

Правила, які треба знати напам'ять

- Локальний префікс отримують **тільки** сторінки в `app/[locale]/`. `/admin`, `/payment`, `/portal`, `/architect` живуть без локалі; на `/uk/admin` middleware сам зробить редірект.
- Додаєш сторінку — **спочатку виріши**: публічна (→ `app/[locale]/...`, переклади в усі 5 файлів `messages/`) чи внутрішня (→ `app/admin|portal|payment|architect/...`). Дерева не взаємозамінні.
- Публічні без логіна: `/admin/login`, `/admin/setup`, `/api/admin/auth/*`, `/api/admin/setup`.
- Middleware ставить заголовки X-Frame-Options: DENY тощо; другий шар (включно з CSP і HSTS) додає `vercel.json`.

Автентифікація, ролі та безпека

Як працює вхід

- 1. Користувач логіниться на `/admin/login` → `/api/admin/auth/*` перевіряє пароль (bcrypt) і, за наявності, TOTP-код 2FA (`lib/totp.ts` , модуль `/admin/2fa`).
- 2. Сервер підписує JWT секретом `JWT_SECRET` (через `jose`) і кладе його в httpOnly-cookie `kompasscrm_session` .
- 3. Далі кожен запит перевіряють **два шари**: `middleware.ts` на рівні маршруту і `requireAuth(["admin","moderator"])` з `lib/auth.js` усередині кожного API-роуту. Це **обов'язковий патерн** для будь-якого нового роуту в `app/api/admin/**` .

Сім ролей

РОЛЬ	ЩО БАЧИТЬ
<code>admin</code>	Все, включно з /architect (LifeOS), налаштуваннями, фінансами
<code>moderator</code>	Майже все операційне, без фінансів і налаштувань
<code>manager</code>	Ліди, угоди, справи, комунікації, операції
<code>sales</code>	Ліди, угоди, скрипти продажів, колл-центр
<code>lawyer</code>	Виконавчі справи, суди, пошта
<code>user</code>	Мінімум: справи клієнтів, інструкція
<code>member</code>	Тільки кабінет /admin/me — middleware жорстко редіректить

ДВІ НЕЗАЛЕЖНІ СИСТЕМИ ПРАВ

Доступ до API контролює `requireAuth([...])` у кожному роуті, а видимість меню — окремо `lib/rbac.js` (масив `NAV` + `navFor(role)`). **Сховати пункт меню ≠ закрити API**. Змінюючи права — онови обидва місця.

Інші механізми безпеки

- **Rate limit**: 100 зап/хв на IP для API, 20 — для auth; публічний чат — окремо 10 зап/хв (`lib/rate-limit.ts`).
- **CSRF**: для не-GET запитів звіряється Origin із Host.
- **CSP + HSTS + X-Frame-Options** — у `vercel.json` .
- **RODO/GDPR**: `lib/rodo.ts` — журнал згод і «право на забуття»; модуль `/admin/rodo` ; RLS-локдаун — міграція 024.

Публічний сайт (5 мов)

Локалізація

- Локалі: `uk` (дефолт), `pl`, `en`, `ru`, `rom` — задані в `i18n.ts`; `localePrefix: "always"` — URL завжди з префіксом (`/uk/...`).
- Усі рядки — в `messages/{locale}.json`; компоненти читають через `useTranslations()`.
- Додаєш рядок — додай у всі 5 файлів**, інакше одна з мов впаде або покаже ключ.

Сторінки `app/[locale]/`

МАРШРУТ	ЩО ЦЕ
<code>/</code>	Головна: Hero, ServicesGrid, HowItWorks, Pricing, Reviews, Team, FAQ, Blog, ContactForm
<code>/pricing</code> · <code>/plans</code>	Прайси та тарифні плани
<code>/karta</code>	«Карта побиту» (нещодавно чинилась — TypeScript + локалі)
<code>/oraku1</code>	Інтерфейс бота Оракул для кандидатів
<code>/book</code>	Запис на консультацію
<code>/manual</code>	Публічна інструкція користувача
<code>/doctrine</code>	Доктрина / маніфест компанії
<code>/privacy</code> · <code>/regulamin</code>	Політика конфіденційності, регламент

Ключові компоненти

ГРУПА	КОМПОНЕНТИ
AI-чат	<code>ChatBot.tsx</code> — публічний чатбот (§ 10.1)
Лідогенерація	<code>ContactForm</code> , <code>SituationQuiz</code> (квіз «ваша ситуація»), <code>ExitPopup</code> , <code>ReturnVisitor</code> , <code>MobileCTABar</code> , <code>AIAssistantIntake</code>
Довіра	<code>Reviews</code> , <code>SocialProof</code> , <code>GuaranteeSection</code> , <code>Team</code> , <code>PortfolioCarousel</code>
Оплата на сайті	<code>PayModal</code> , <code>PaymentForm</code> , <code>P24PaymentSteps</code>
Атмосфера	<code>StarField</code> , <code>CosmicSpiral</code> , <code>SpotlightCard</code> , <code>ScrollProgress</code>
Тема	<code>ThemeSwitch</code> / <code>ThemeToggle</code> + <code>lib/ThemeContext.tsx</code> : темна/світла, <code>localStorage</code> -ключ <code>theme</code> , атрибут <code>data-theme</code> на <code><html></code>

Адмінка / CRM — два паралельні світи

Найпідступніший факт проєкту: всередині `/admin` живуть дві окремі, паралельні CRM-реалізації — і обидві робочі:

	APP/ADMIN/(PANEL)/*	APP/ADMIN/CRM/*
Кількість модулів	~115 папок	~27 папок
API	<code>/api/admin/*</code>	<code>/api/admin/crm/*</code>
Приклад	<code>/admin/leads</code>	<code>/admin/crm/leads</code>
Походження	Масова генерація преміум-UI (PROJECT.md)	Пізніша, компактніша CRM

`/admin/leads` і `/admin/crm/leads` — це **різні сторінки з різними API**. Перед редагуванням завжди перевір, у якому дереві живе модуль.

ГОЛОВНА ПАСТКА — МОКОВІ МОДУЛІ

Більшість модулів (`panel`) згенеровані як красивий каркас із фейковими даними. У кожному є «AI-консоль агента» (`components/admin/AgentConsole.jsx`), яка друкує **симульовані** лог-рядки через `setInterval`. Це декорація, не бекенд. Якщо модуль виглядає повністю робочим — це ще нічого не значить: перевір, чи існує API-роут, який він викликає, і чи торкається той роут реальних таблиць.

Реально працюючі напрямки (живі API та таблиці): ліди, угоди, задачі, справи, виконавчі провадження, пошта, шаблони документів, RODO, звіти, учасники профспілки, платежі/підписки, автоматизації.

Групи модулів (за меню `lib/rbac.js`)

ГРУПА МЕНЮ	МОДУЛІ
Продажі та Клієнти	members, leads, deals, leads-finder, client-portal, loyalty, playbooks
Справи та Юриспруденція	cases, enforcement, work-permits, litigation, contracts, e-signatures, doc-builder
Комунікація та Маркетинг	emails, email-sequences, messengers, livechat, broadcasts, call-center, marketing, forms
Фінанси та Аналітика	reports, accounting, e-invoicing (KSeF), expenses, subscriptions, currencies
Операції та Логістика	appointments, booking, insurance (ZUS), housing, fleet, mailroom, hr-leave, service-catalog
Компанія та Розвиток	academy, knowledge-base, gamification, partner-portal, referrals
ШІ та Інструменти	copilot, ocr-scanner, workflows, gov-integration

Система	/architect (LifeOS), manual, settings, integrations, 2fa
---------	--

Спільні UI-примітиви

`components/admin/ui.jsx` : StatCard, ProgressBar, DataTable, EmptyState, SearchInput, іконки PATHS. Каркаси: Shell.jsx / DualSidebarShell.jsx. Також: KanbanBoard, CommandPalette, GlobalSearch, NotificationCenter, ImportWizard, FileUpload, Timeline, LeadDetailsModal, AiCopilotSidebar.

База даних — два клієнти, 31 міграція

Два способи доступу — не взаємозамінні

	LIB/DB.JS (RAW PG)	LIB/SUPABASE.TS (SUPABASE JS)
Експортує	<code>q(text, params) , one(text, params)</code>	<code>supabase (anon), supabaseAdmin (service key)</code>
З'єднання	Новий <code>pg.Client</code> на кожен запит, без пулу	HTTP-клієнт Supabase
Хто використовує	Майже всі <code>/api/admin/*</code> , CRM-таблиці (<code>leads, tasks, deals, Kompas_*</code>)	Агенти (<code>agents, agent_tasks, god_policies</code>), частина бот-лідів
Конфіг	Автовизначення: <code>PGHOST</code> → <code>DATABASE_URL</code> → <code>POSTGRES_URL</code> → <code>POSTGRES_HOST</code> → <code>localhost</code>	<code>NEXT_PUBLIC_SUPABASE_URL</code> + ключі

Пишеш новий роут — подивись, у яких таблицях дані, і бери той самий клієнт, що й сусідні роути цього домену.

Міграції `db/migrations/` (001–031)

Нова міграція = наступний номер. Раннера немає — застосовуються вручну:

```
psql "$DATABASE_URL" -f db/migrations/0NN_nazva.sql
# або: scripts/apply-migration-file.ts
```

№№	ТЕМА
001–007	Ліди (кошик, оплати), канбан працівників, задачі, автоматизації, growth
008–009	Поля Оракула, імміграційні справи клієнтів
010–015	Угоди, активності, файли, звіти, e-mail, виконавчі провадження
016–018	LifeOS: ядро, двигуни/логи, монетизація/CFO
019–022	Цифровий профіль профспілки, працевлаштування, легалізація, маркетплейс партнерів
023–025	Email-кампанії, RLS-локдаун безпеки , дзвінки CRM + налаштування
026–028	Мілена : sales-бот, стартові knowledge cards, RLS для Мілени
029–031	Веб-сесії Оракула, статуси <code>Kompas_leads</code> (check + dropped)

`db/schema.sql` — базова схема (старі `Kompas_*` таблиці); `supabase/*.sql` — те, що застосовували напряду в Supabase.

AI-підсистеми — чотири боти та LifeOS

П'ять незалежних систем. Вони не діляться кодом і промптами — не плутати.

10.1 · Публічний чатбот (сайт)

- Роут `POST /api/chat`; модель Gemini (`ai` SDK); UI — `components/ChatBot.tsx`.
- Tool-calling: пошук вакансій (`kompas_jobs_v2`), партнерів (`kompas_partners`); інструмент `record_lead` пише ліди в `kompas_leads`.
- Rate limit 10 зап/хв/IP; підтримка локальної LLM: `USE_LOCAL_LLM` + `LOCAL_LLM_URL/MODEL`.

10.2 · Оракул — кандидати

- Роут `POST /api/orakul/chat`; код `lib/orakul-bot.ts` + `lib/orakul-prompt.ts`; UI — сторінка `/[locale]/orakul`.
- Окрема персона і промпт. Веде кандидатів на працевлаштування, трекає веб-сесії (міграція 029).
- Cron `/api/cron/orakul-abandoned` (щодня 06:00) «підбирає» покинуті діалоги.

10.3 · Оракул — роботодавці (EWU-рекрутинг)

- Код `lib/orakul-employer.ts`: структурована анкета роботодавця (компанія, NIP, позиції, ставки, житло, сертифікати зварювальників тощо).
- Принцип: критичні правила живуть **у детермінованому коді, а не в промпті** — витяг JSON, рендер даних і тригери ескалації (кількість людей / ставка) продубльовані кодом як бекстоп до LLM.
- Нотифікації: `EMPLOYER_LEAD_NOTIFY_EMAIL`.

10.4 · Мілена — бот продажів (месенджери)

- `lib/milena-bot.ts` — **детермінований рушій**: інтенти, обов'язкові поля і мета-флоу вирішують «ЩО робити». Claude викликається лише в `app/api/bot/milena/message/route.ts` — «ЯК сказати».
- Передача людині: `lib/milena-handoff.ts`. Дані: міграції 026–028. Тести: `lib/__tests__/milena-bot.test.ts`.

10.5 · God / Primus — оркестрація агентів

- `POST /api/god/command` → `POST /api/agents/primus/dispatch` → задачі в Supabase-таблицях `agents` / `agent_tasks` (`lib/agents.ts`, `lib/god.ts`, політики — `god_policies`).
- Монітор heartbeat'ів: `GET /api/agents/monitor/cron` + алерти через `lib/notify.ts`.
- UI: `/admin/agents` (AgentsDashboard → GodCard + AgentCard[], SWR-пулінг 10 с).

ВІДОМИЙ БОРГ

Авторизація God/Primus — хардкод е-mail (iphoenixgsm@gmail.com), не полі.

10.6 · LifeOS — окрема система!

lib/lifeos/ (alexDigital, fateEngine, soulEngine); UI — /architect/* (тільки admin); cron /api/cron/lifeos щодня опівночі. **Не плутати з God/Primus** — спільна лише тема «автономних агентів», код незалежний.

Платежі — три провайдери

`/api/payment` пробує провайдерів по черзі — бере перший, для якого налаштовані env-змінні:

1 · ОСНОВНИЙ
Przelewy24

2 · РЕЗЕРВ
PayU

3 · РЕЗЕРВ
Stripe

4 · FALLBACK
Мок
`/payment/mock/[sessionId]`

ПРОВАЙДЕР	КЛІЄНТ	ВЕБХУК	ОСОБЛИВОСТІ
Przelewy24	<code>lib/przelewy24.ts</code>	<code>/api/payment-notify</code> (IPN)	Підпис SHA-384; sandbox через <code>P24_SANDBOX</code>
PayU	<code>lib/payu.ts</code>	<code>/api/payu/notify</code>	
Stripe	<code>lib/stripe.ts</code>	<code>/api/stripe/webhook</code>	+ <code>/api/stripe/checkout-architecture</code>

Сторінки оплат — `app/payment/*` (без локального префікса). Після успішної оплати: `lib/commissions.ts` рахує комісію агента за правилами `kompas_commission_rules`, `lib/invoices.ts` — інвойси.

ПАМ'ЯТАЙ

Провайдери не уніфіковані спільним інтерфейсом. Перш ніж чіпати платіжний код — з'ясуй, до якої пари «lib + вебхук» він належить.

Месенджери та вебхуки

Усе вхідне — під `app/api/bot/*`:

КАНАЛ	КЛІЄНТ	ВЕБХУК / ПРИМІТКИ
Telegram	<code>lib/telegram.ts</code>	<code>/api/bot/webhook</code> + <code>/api/bot/setup</code> (реєстрація; секрети <code>TELEGRAM_WEBHOOK_SECRET</code> / <code>TELEGRAM_SETUP_SECRET</code>)
WhatsApp	<code>lib/whatsapp.ts</code>	<code>/api/bot/whatsapp</code> (<code>WHATSAPP_TOKEN</code> , <code>PHONE_ID</code> , <code>VERIFY_TOKEN</code>)
Viber	—	<code>/api/bot/viber-webhook</code>
Facebook	—	<code>/api/bot/fb-webhook</code>
Мілена	<code>\$ 10.4</code>	<code>/api/bot/milena/message</code>
Вихідні	—	<code>/api/bot/outbound</code> — закритий авторизацією після security-фіксу

Адмін-алерти летять у Telegram (`TELEGRAM_ADMIN_CHAT_ID` — найуживаніша env-змінна проєкту) та на пошту через `lib/notify.ts` / `lib/email.ts` (SendGrid). Вхідна пошта — `imapflow`; HTML вхідних листів санітизується (був окремий security-фікс).

Cron-завдання

Роутів у `app/api/cron/` — **дев'ять**, але у `vercel.json` зареєстровано **лише два**:

РОУТ	У VERCEL.JSON	РОЗКЛАД	ЩО РОБИТЬ
<code>/api/cron/lifeos</code>	так	<code>0 0 * * *</code>	Щоденний цикл LifeOS
<code>/api/cron/orakul-abandoned</code>	так	<code>0 6 * * *</code>	Покинуті діалоги Оракула
<code>/api/cron/appointment-reminders</code>	ні	—	Нагадування про записи
<code>/api/cron/daily-snapshot</code>	ні	—	Щоденний зріз метрик
<code>/api/cron/dues-reminders</code>	ні	—	Нагадування про внески
<code>/api/cron/lead-followup</code>	ні	—	Фолоу-ап лідів
<code>/api/cron/nps-survey</code>	ні	—	Розсилка NPS
<code>/api/cron/subscription-renewal</code>	ні	—	Продовження підписок
<code>/api/cron/weekly-digest</code>	ні	—	Тижневий дайджест

ВИСНОВОК ДЛЯ ШЕФА

Сім кронів написані, але **ніколи не запускаються самі** — їх треба або додати у `vercel.json`, або смикати зовнішнім планувальником. Усі захищені секретом `CRON_SECRET`. Окремо живе монітор агентів: `GET /api/agents/monitor/cron`.

Портали: клієнт, учасник, NPS

ДЕРЕВО	ХТО ЗАХОДИТЬ	ЩО ТАМ
<code>app/portal/</code> <code>/portal/case/...</code> · <code>/portal/project/...</code>	Клієнт за посиланням	Статус своєї справи/проєкту без доступу до CRM
<code>app/member/</code> <code>jobs</code> · <code>legal</code> · <code>marketplace</code> · <code>profile</code> · <code>rewards</code>	Учасник профспілки	Вакансії, юридопомога, маркетплейс, профіль, бонуси
<code>app/admin/me</code>	Роль member у CRM	Особистий кабінет (middleware пускає member'a тільки сюди)
<code>app/nps/</code>	Клієнт з розсилки	Оцінка NPS (<code>/api/nps</code>)
<code>app/architecture/</code>	Публічно	Презентаційна сторінка архітектури (+ Stripe-checkout)

Тестування та CI

Юніт-тести (Jest)

- Живуть **тільки** в `__tests__/` та `lib/__tests__/` — конфіг ловить шаблон `**/__tests__/**/*.test.{ts,tsx}`. Тест поза цими папками просто не запуститься.
- Покрито ядро: `agents`, `god`, `monitor`, `notify`, `milena-bot`, `orakul-employer`, `chat-route`, `payment-route`, `rate-limit`, `routes.integration`.

E2E (Playwright, `e2e/*.spec.ts`)

`agents.spec.ts` · `main-site-verify.spec.ts` · `orakul-verify.spec.ts` · `orakul-candidate-and-metadata.spec.ts`. Команда `pnpm test:e2e` сама піднімає дев-сервер.

ЧОГО ТЕСТИ НЕ ГАРАНТУЮТЬ

CI — заглушка (§ 3), збірка не перевіряє типи. Реальна перевірка перед релізом: `pnpm lint && pnpm typecheck && pnpm test:unit`, для критичних змін — плюс `e2e`.

Змінні середовища (ENV)

ГРУПА	ЗМІННІ
Ядро	JWT_SECRET (без нього прод не стартує), NEXT_PUBLIC_APP_URL , NODE_ENV
БД (пріоритет зліва направо)	PGHOST/PGUSER/PGPASSWORD/PGDATABASE/PGPORT/PGSSL → DATABASE_URL → POSTGRES_URL → POSTGRES_HOST...
Supabase	NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY (варіанти: SUPABASE_SERVICE_ROLE, SUPABASE_SERVICE_KEY)
AI	GEMINI_API_KEY (чатбот/Оракул), ANTHROPIC_API_KEY (Мілена), USE_LOCAL_LLM + LOCAL_LLM_URL + LOCAL_LLM_MODEL
Przelewy24	P24_MERCHANT_ID, P24_CRC, P24_API_KEY, P24_SANDBOX
PayU / Stripe	PAYU_*, PAYU_SANDBOX; STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET
Telegram	TELEGRAM_ADMIN_CHAT_ID , TELEGRAM_BOT_TOKEN, TELEGRAM_BOT_TOKENS, TELEGRAM_WEBHOOK_SECRET, TELEGRAM_SETUP_SECRET, ADMIN_TELEGRAM_CHAT_ID
WhatsApp	WHATSAPP_TOKEN, WHATSAPP_PHONE_ID, WHATSAPP_VERIFY_TOKEN
Пошта	SENDGRID_API_KEY, RESEND_API_KEY
Крони	CRON_SECRET
Інше	EMPLOYER_LEAD_NOTIFY_EMAIL

ПАТЕРН CLEANENV — ОБОВ'ЯЗКОВИЙ

Env-змінні, вставлені з Windows/PowerShell, часто тягнуть невидимий BOM (U+FEFF) та `\r`. Тому `middleware.ts`, `lib/db.js`, `lib/auth.js` читають env через хелпер `cleanEnv` / `clean`. Читаєш нову «сиру» env-змінну в серверному коді — використовуй той самий хелпер, не вигадуй свій.

Довідник бібліотек lib/

ФАЙЛ	ПРИЗНАЧЕННЯ
auth.js	requireAuth(roles) — перевірка JWT у API-роутах
rbac.js	Масив NAV (меню) + navFor(role)
db.js	q(), one() — raw Postgres
supabase.ts	Клієнти supabase / supabaseAdmin
rate-limit.ts	Ліміт запитів (чат)
totp.ts	TOTP 2FA — генерація і перевірка кодів
rodo.ts	GDPR: журнал згод, право на забуття
email.ts · notify.ts	Вихідна пошта (SendGrid), алерти адмінам
telegram.ts · whatsapp.ts	Клієнти месенджерів
przelewy24.ts · payu.ts · stripe.ts	Платіжні клієнти
invoices.ts · commissions.ts	Інвойси; комісії агентів після оплат
orakul-bot.ts · orakul-prompt.ts	Оракул-кандидати: логіка + промпт
orakul-employer.ts	Оракул-роботодавці (EWU): JSON-екстракція, ескалації
milena-bot.ts · milena-handoff.ts	Мілена: детермінований рушій + передача людині
agents.ts · god.ts · monitor.ts	God/Primus: агенти, політики, heartbeat-монітор
lifeos/*	LifeOS: alexDigital, fateEngine, soulEngine
task-from-lead.ts	Автостворення задачі з ліда
template-render.js	Рендер шаблонів документів
doctrine.ts	Контент сторінки «Доктрина»
navigation.ts	Локалізована навігація публічного сайту
ThemeContext.tsx · useCookieConsent.ts	Тема; згода на cookies

Рецепти: як робити типові задачі

Додати публічну сторінку

1. Створи `app/[locale]/nova-storinka/page.tsx` (**TypeScript** — публічне дерево на TS).
2. Усі тексти — через `useTranslations()`; ключі — в усі 5 файлів `messages/*.json`.
3. Перевір на `/uk/nova-storinka` і ще хоча б одній локалі.

Додати модуль в адмінку

1. Виріши дерево: `(panel)` чи `crm` (§ 8) — і роби в стилі сусідів (**JS/JSX** в адмінці).
2. Сторінка: `app/admin/(panel)/modul/page.jsx`; API: `app/api/admin/modul/route.js`.
3. У роуті **першим ділом** — `requireAuth(...)` з `lib/auth.js`.
4. Пункт меню — в `NAV` у `lib/rbac.js` з правильними `roles`.
5. Дані — через `q()` / `one()` з `lib/db.js` (якщо це CRM-таблиці).

Додати міграцію БД

1. Файл `db/migrations/032_nazva.sql` (наступний номер!).
2. Застосуй: `psql "$DATABASE_URL" -f db/migrations/032_nazva.sql`.
3. Для Supabase-таблиць не забудь RLS (зразки — міграції 024 і 028).

Перед комітом — завжди

```
pnpm lint && pnpm typecheck && pnpm test:unit
```

ДЕПЛОЙ

Push у `main` = деплой на продакшн. Без винятків.

Граблі, на які вже наступали

1. **Зелений build ≠ здоровий код** — типи й лінт вимкнені у збірці (§ 3).
2. **CI — заглушка**. Галочка на GitHub нічого не перевіряє.
3. **Два дерева CRM** — переконайся, що редагуєш правильне (§ 8).
4. **Мокові модулі** — красивий UI ≠ робочий бекенд; «телеметрія» AI-консолей — симуляція.
5. **Два клієнти БД** — pg та Supabase не взаємозамінні (§ 9).
6. **JS vs TS — навмисно**. Адмінка на JS, публічний сайт на TS. Не «виправляй» розширення — підлаштовуйся під файл, який редагуєш.
7. **BOM в env** — читай env лише через cleanEnv (§ 16).
8. **README.md застарілий** (4 локалі замість 5; немає Stripe/PayU/LifeOS/Оракула). Джерело правди — CLAUDE.md, ця методичка і сам код. У CLAUDE.md теж є розбіжності: дефолтна локаль насправді uk (не pl), міграцій 31 (не 22).
9. **God/Primus авторизується хардкод-email'ом**, не ролями — відомий борг.
10. **7 із 9 кронів не заплановані** у vercel.json (§ 13).
11. **scripts/_archive/** — одноразові історичні скрипти, не зразок для нових. **.agents/** — лог минулих агент-сесій, не документація. **docs/agent-learnings.md** — append-only, руками історію не правити.
12. **Локаль і адмінка несумісні**: /uk/admin не існує — посилання на admin/payment/portal/architect завжди пиши без локалі.

Де шукати додаткову документацію

ДОКУМЕНТ	ЩО ВСЕРЕДИНІ
CLAUDE.md	Технічний гід для AI-асистентів — найактуальніший огляд архітектури
docs/crm_manual_uk.md	Інструкція користувача CRM (для співробітників)
docs/website_manual_uk.md	Інструкція користувача сайту (для клієнтів)
PROJECT.md · ORIGINAL_REQUEST.md	Історія генерації 120-модульної адмінки
docs/agent-learnings.md	Журнал висновків автоматичних аудитів
db/schema.sql · db/migrations/ · supabase/*.sql	Повна історія схеми даних

Методичку складено автоматично на основі аудиту живого коду репозиторію KompasMіgrасії станом на 22.07.2026. При розбіжностях між документом і кодом — правий код; онови методичку. · Kompas Mіgrасії — DOMUS V Sp. z o.o.